SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1402 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

IHARI CHARAN P(192511107) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:HARI CHARAN P

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

1. Banking Transaction Processing System

A banking software company is developing a compiler for a domainspecific scripting language used in transaction processing systems. The compiler must translate variable declarations, assignment statements, and Boolean expressions efficiently to avoid transaction failures during peak hours. The system also uses case statements to classify transactions such as deposit, withdrawal, and loan processing. During testing, incorrect intermediate code generation caused delays in transaction execution.

Tasks:

(i) Design suitable intermediate code for the given assignment and Boolean expressions. *(K3)*

(ii) Explain how backpatching can help in handling conditional transaction validation. *(K4)*

(iii) Analyze the role of intermediate languages in improving banking software portability. *(K5)*

2. Smart Traffic Control System

An automotive company is building a smart traffic management application where signals are controlled automatically based on traffic density. The compiler used in the embedded controller must process Boolean expressions and case statements for different traffic conditions. Due to inefficient intermediate code generation, the response time of traffic signals increased significantly during rush hours. The development team plans to optimize procedure calls and assignment statements in the compiler design.

Tasks:

(i) Construct intermediate code for the traffic signal control logic. *(K3)* (ii) Explain how procedure calls improve modularity in the traffic control application. *(K4)*

(iii) Evaluate the importance of compiler optimization in real-time embedded systems. *(K5)*

3. E-Commerce Product Recommendation Engine

An e-commerce company uses a rule-based recommendation engine developed using a custom programming language. The compiler processes declarations, assignment statements, and Boolean conditions to generate recommendations dynamically. During seasonal sales, delays occurred because of improper handling of case statements and inefficient intermediate code generation. The organization wants to redesign the compiler to improve execution speed and maintainability.

Tasks:

(i) Generate intermediate representation for the recommendation logic. *(K3)*

(ii) Discuss how case statements can efficiently handle product category selection. *(K4)*

(iii) Analyze the role of compiler design in improving large-scale ecommerce applications. *(K5)*

4. Hospital Patient Monitoring System

A healthcare technology company is developing a patient monitoring system that continuously checks vital signs using embedded software. The compiler used in the system must efficiently evaluate Boolean expressions and assignment statements for emergency alerts. Improper backpatching in conditional branching caused delayed alarm generation during critical patient conditions. The software team is redesigning the compiler for faster and more reliable intermediate code execution.

Tasks:

(i) Design intermediate code for patient condition monitoring logic. *(K3)* (ii) Explain the significance of backpatching in emergency alert systems.

(K4)

(iii) Evaluate how compiler optimization improves reliability in healthcare applications. *(K5)*

5. Online Airline Reservation System

An airline company is modernizing its reservation platform using a custom scripting language for ticket booking and seat allocation. The compiler must handle declarations, assignment statements, and procedure calls efficiently to manage thousands of booking requests simultaneously. During testing, errors in intermediate code generation affected seat allocation and payment confirmation processes. The software architects aim to improve compiler performance for large-scale distributed applications.

Tasks:

(i) Develop intermediate code for ticket booking and seat allocation operations. *(K3)*

(ii) Explain how procedure calls support modular reservation management. *(K4)*

(iii) Analyze the role of intermediate languages in distributed airline reservation systems. *(K5)*

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 25**	**/ 20**	**/ 20**	**/ 10**	**/ 100**

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Question 1 — Banking Transaction Processing System

Tasks: *Design intermediate code for assignment/Boolean expressions (K3); explain how backpatching helps conditional transaction validation (K4); analyze the role of intermediate languages in banking software portability (K5).*

Task (i): Intermediate Code for Assignment & Boolean Expressions (K3)

Consider a representative withdrawal-validation statement from the transaction-processing script:

```
balance = 5000;
amount  = 1500;

if (amount <= balance && amount > 0) then
    balance = balance - amount;
    status  = "SUCCESS";
else
    status  = "FAILED";
```

The compiler uses a Syntax-Directed Translation (SDT) scheme in which the Boolean expression B is annotated with two synthesized attributes, **B.truelist** and **B.falselist**, holding lists of incomplete jump quadruples. For the AND operator (B → B1 && M B2), the grammar rule backpatches B1.truelist to the instruction right after B1 (marker M, which records the next quad number), so control falls through to evaluate B2 only when B1 is true — this is exactly what implements short-circuit evaluation. The resulting one-pass, single-scan Three-Address Code (TAC) is:

Op	Arg1	Arg2	Result
<=	amount	balance	t1
if t1	goto		L2 (100)
goto			L5 (105)
>	amount	0	L2: t2
if t2	goto		L3 (103)
goto			L5 (105)
−	balance	amount	L3: t3

=	t3		balance
=	"SUCCESS"		status
goto			L4
=	"FAILED"		L5: status

Each quad is numbered so that jump targets (100, 103, 105 above) are concrete addresses; in one-pass generation these targets are initially left blank and filled in later via **backpatch()** — which is exactly the mechanism analyzed in Task (ii).

Architecture / Flow Diagram

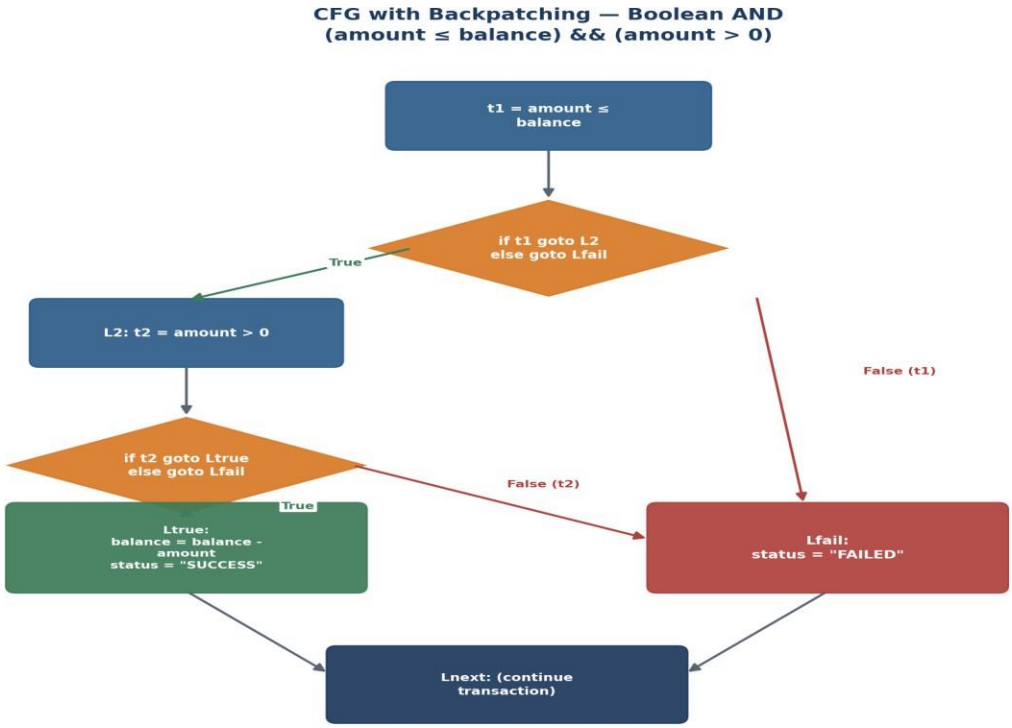

Figure 1. Control-flow graph showing backpatched true/false edges for the AND condition guarding the withdrawal.

Task (ii): Role of Backpatching in Conditional Transaction Validation (K4)

Backpatching is a one-pass code-generation technique that allows the compiler to generate jump (goto) instructions before their target labels are known, by initially leaving the target field of a quad empty and recording its quad number in a list (truelist or falselist). Three helper functions drive this:

- **makelist(i)** — creates a new list containing only quad number i (a single incomplete jump).
- **merge(p1, p2)** — concatenates two lists, used when a false-list from one sub-condition must be combined with another (e.g., chaining multiple validation checks such as sufficient balance, valid account status, and fraud-score threshold).
- **backpatch(list, target)** — inserts the target address into every quad referenced in the list, once that target becomes known (e.g., once the code for the "SUCCESS" or "FAILED" branch has been generated).

In the banking scenario, a transaction is validated by a conjunction of conditions — sufficient balance, non-negative amount, valid account state, and possibly a fraud-check flag. Backpatching lets the compiler emit each comparison and its conditional jump immediately as it parses the expression left to right, without needing to pre-scan the entire condition or invent placeholder labels for every sub-expression. Only when the "true branch" (debit + SUCCESS) and "false branch" (FAILED) code blocks are actually generated does the compiler patch in their real addresses. This is precisely why the assessment scenario's incorrect intermediate code generation caused transaction delays: without correct backpatching, jump targets can point to the wrong quad, causing a validated transaction to be wrongly rejected (or, worse, an invalid one to be approved) — both are severe correctness bugs in a financial system. Correct backpatching guarantees each conditional branch resolves to exactly the intended validation path in a single compilation pass, which keeps compile time low and is essential when scripts are re-compiled or hot-reloaded during peak trading/transaction hours.

Task (iii): Role of Intermediate Languages in Banking Software Portability (K5)

An intermediate representation (IR) such as Three-Address Code sits between the source scripting language and machine-specific target code, and this separation is what makes portability possible and analyzable:

- **Machine independence:** TAC makes no assumptions about registers, addressing modes, or instruction sets, so the same front end (lexer, parser, semantic analyzer, IR generator) can be reused unchanged when the bank migrates from on-premise mainframes to cloud-based microservice runtimes — only the back-end code generator needs to be retargeted.
- **Reusable, machine-independent optimization:** Optimizations such as constant folding, common sub-expression elimination, and dead-code elimination operate directly on the IR once and benefit every deployment target, rather than needing to be re-implemented per platform.
- **Easier verification and auditability:** Because IR is simpler and more regular than either source syntax or machine code, it is easier to statically analyze for regulatory/compliance checks (e.g., proving that every withdrawal path enforces a balance check) — an important requirement in banking software audits.
- **Multi-target deployment:** A single script (e.g., a fraud-rule or transaction-classification script) can be compiled once to IR and then emitted for multiple execution environments — ATMs, core-banking mainframes, and mobile-banking backends — ensuring identical business logic and reducing the risk of divergent behaviour across channels.
- **Long-term maintainability:** As banking hardware evolves (new processors, cloud accelerators), only the back end needs updating; the IR and all front-end investment remain valid, protecting the compiler's long-term value.

Question 2 — Smart Traffic Control System

Tasks: *Construct intermediate code for the traffic signal control logic (K3); explain how procedure calls improve modularity (K4); evaluate the importance of compiler optimization in real-time embedded systems (K5).*

Task (i): Intermediate Code for Traffic Signal Control Logic (K3)

The controller classifies traffic density into LOW / MEDIUM / HIGH (with a DEFAULT fallback) using a case statement to select a signal-green duration:

```
case density of
   LOW:     signal = 10;
   MEDIUM:  signal = 25;
   HIGH:    signal = 45;
   default: signal = 20;
end case
```

Because the case values (0,1,2) are small, dense integers, the compiler generates a computed (indexed) jump through a jump table rather than a sequence of comparisons — this gives O(1) dispatch instead of O(n), which matters directly for the embedded controller's rush-hour response-time requirement. The TAC is:

Op	Arg1	Arg2	Result
=	density		t1
>=	t1	0	t2
if t2 == 0	goto		Ldefault
<=	t1	2	t3
if t3 == 0	goto		Ldefault
goto	*	(JumpTable + t1)	
L1: =	10		signal
goto			Lend
L2: =	25		signal
goto			Lend
L3: =	45		signal
goto			Lend

Ldefault: =	20		signal

Call-site TAC:

```
param density
t1 = call getSignalTiming, 1
```

The instruction “goto *(JumpTable + t1)” is a computed/indexed jump: JumpTable[0]=L1, JumpTable[1]=L2, JumpTable[2]=L3 — the compiler builds this table at compile time from the case labels.

Architecture / Flow Diagram

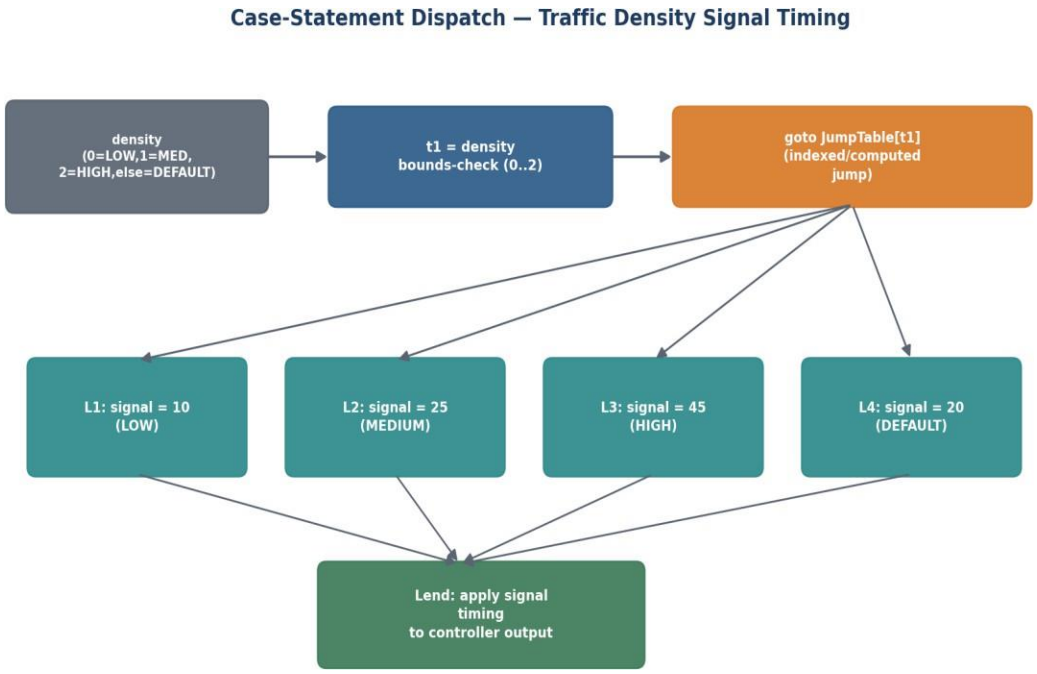

Figure 2. Case-statement jump-table dispatch for traffic-density signal timing.

Task (ii): Procedure Calls Improve Modularity in Traffic Control (K4)

Rather than inlining the density-classification and timing logic at every intersection controller, the redesigned compiler factors it into a reusable procedure, e.g. getSignalTiming(density). Its call generates standard TAC call-sequence instructions:

- **Encapsulation & reuse:** The same compiled procedure is invoked by every intersection's controller instance, instead of the compiler duplicating the case-dispatch code at each call site — reducing code size, which matters on memory-constrained embedded microcontrollers.

- **Independent optimization:** Because the logic exists in exactly one place, compiler optimizations (jump-table dispatch, constant propagation) are applied once and benefit every intersection, rather than needing to be re-verified at each inlined copy.
- **Easier testing & maintenance:** Traffic engineers can unit-test getSignalTiming() in isolation and update timing thresholds (e.g., for a new HIGH-density definition) in a single procedure, without touching every call site across the city's signal network.
- **Clean activation-record management:** Parameters (density) and return values (signal duration) are passed via a well-defined activation record, so the procedure can be safely called concurrently from multiple intersection controllers without interference.

Task (iii): Importance of Compiler Optimization in Real-Time Embedded Systems (K5)

Real-time traffic control has a hard timing budget: the controller must sense density and update signal state well within the inter-vehicle arrival interval, or congestion compounds. Compiler optimization is not a "nice-to-have" here — it directly determines whether the system meets its deadlines:

- **Worst-Case Execution Time (WCET) reduction:** Optimizations such as jump-table dispatch (used above, O(1) vs. O(n) if-else chains), constant folding, and redundant-comparison elimination shrink the worst-case path length, which is what real-time schedulability analysis actually cares about — not just average-case speed.
- **Dead-code and common-subexpression elimination:** Removes redundant density re-reads or duplicate bounds checks across nested conditions, directly cutting instruction count on the interrupt-driven control loop.
- **Register allocation:** Keeps frequently-used variables (density, signal) in registers rather than memory, reducing per-iteration latency and jitter — critical because inconsistent response time (jitter), not just average speed, causes unpredictable signal behaviour.
- **Code-size optimization:** Embedded traffic controllers have limited flash/RAM; a smaller optimized binary leaves headroom for safety checks and diagnostics.
- **Trade-off awareness:** Aggressive optimization (e.g., heavy loop unrolling) can increase code size and hurt instruction-cache predictability, so embedded compiler flags are typically tuned for balanced, predictable WCET rather than maximum .

Question 3 — E-Commerce Product Recommendation Engine (10 Marks)

Tasks: *Generate intermediate representation for the recommendation logic (K3); discuss how case statements can efficiently handle product category selection (K4); analyze the role of compiler design in improving large-scale e-commerce applications (K5).*

Task (i): Intermediate Representation for Recommendation Logic (K3)

A representative discount/recommendation rule evaluated per product view:

```
if (category == "electronics" && price > 1000) then
    discount = 0.10;
else if (category == "clothing" && season == "sale") then
    discount = 0.20;
else
    discount = 0.05;
```

Using the same truelist/falselist SDT scheme as Task 1, nested conditions translate into chained comparisons with backpatched jumps. The TAC (quadruples) is:

Op	Arg1	Arg2	Result
==	category	"electronics"	t1
if t1	goto		L1 (105)
goto			L2 (108)
>	price	1000	L1: t2
if t2	goto		L3 (110)
goto			L2 (108)
==	category	"clothing"	L2: t3
if t3	goto		L4 (113)
goto			L6 (118)
==	season	"sale"	L4: t4
if t4	goto		L5 (115)
goto			L6 (118)
=	0.10		L3: discount
goto			Lend

=	0.20		L5: discount
goto			Lend
=	0.05		L6: discount

Note how the false-list of the first AND condition (t1 or t2 failing) is merged and backpatched to L2, the entry point of the second rule — this chaining is what lets an arbitrary number of recommendation rules be added without restructuring earlier code.

Architecture / Flow Diagram

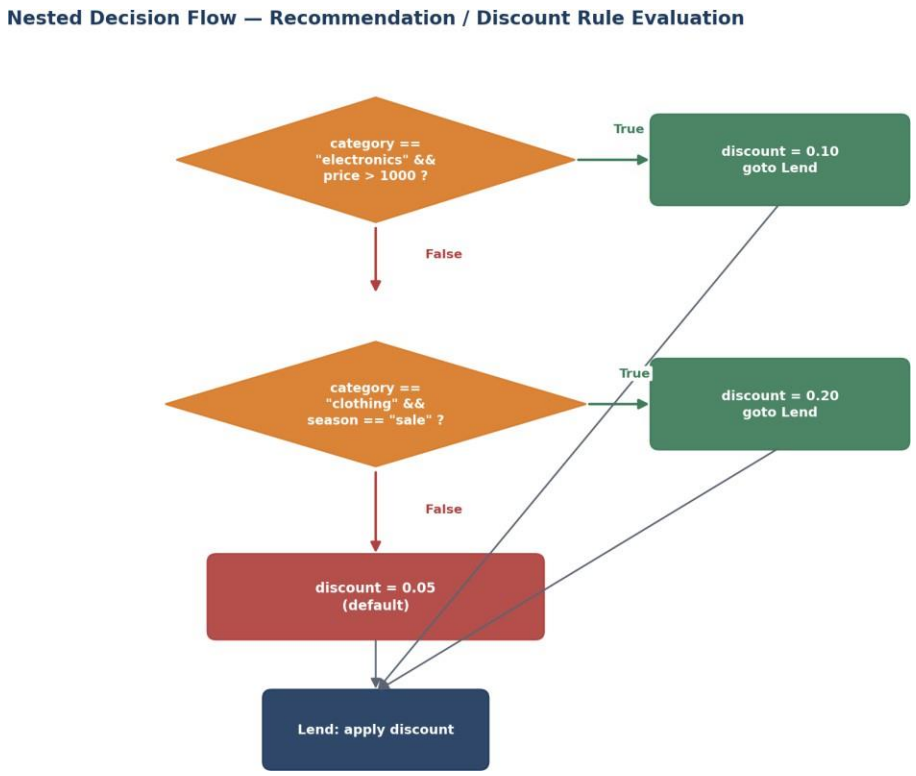

Figure 3. Nested decision flow for category-and-condition-based discount/recommendation selection.

Task (ii): Case Statements for Efficient Product Category Selection (K4)

The original delays during seasonal sales were traced to treating category selection as a long chain of if-else comparisons — an O(n) linear scan through every category before reaching the correct one. Replacing this with a proper case/switch construct gives the compiler two efficient dispatch strategies depending on the category set:

- **Dense case values (jump table):** If categories map to small contiguous integer codes (electronics=0, clothing=1, footwear=2, …), the compiler builds a jump table exactly

as in Q2, giving O(1) dispatch regardless of how many categories exist — essential when the catalogue has hundreds of categories during a sale event.

- **Sparse or string-valued cases (search-based dispatch):** If category labels are non-contiguous or string-valued, the compiler instead generates a balanced binary decision tree of comparisons (via hashing or a sorted comparison tree), giving O(log n) dispatch — still far better than a naive linear if-else chain.
- **Single-evaluation semantics:** The case selector expression (category) is evaluated exactly once and reused for the whole dispatch, whereas the original if-else chain the company used may have implicitly re-evaluated equivalent sub-expressions per branch, adding unnecessary overhead exactly when traffic (and category diversity) was highest.
- **Maintainability:** Adding a new product category becomes a single new case label rather than inserting another link in a growing if-else chain, directly addressing the redesign goal of improved maintainability stated in the scenario.

Task (iii): Role of Compiler Design in Large-Scale E-Commerce Applications (K5)

Recommendation logic is evaluated on nearly every page view/product impression, so even small per-evaluation inefficiencies compound massively under sale-season load. Good compiler design has a direct, measurable business impact:

- **Latency reduction at scale:** Efficient IR generation and case-dispatch (O(1)/O(log n) instead of O(n)) directly cuts the per-request evaluation time of recommendation rules, which is critical when millions of concurrent shoppers are being scored simultaneously during flash sales.
- **Optimizations compounding at scale:** Techniques such as inlining small helper rules, common-subexpression elimination (avoiding re-fetching the same product attribute across multiple rule checks), and constant folding of static thresholds all reduce CPU cost per evaluation — multiplied across millions of daily evaluations, this becomes a significant infrastructure cost and latency saving.
- **Scalability & maintainability trade-off:** A well-designed compiler pipeline (clean IR, modular case dispatch) lets the engineering team add or A/B-test new recommendation rules quickly without re-architecting the whole engine, supporting the business need for frequent rule updates during different sale campaigns.

Question 4 — Hospital Patient Monitoring System

Tasks: *Design intermediate code for patient condition monitoring logic (K3); explain the significance of backpatching in emergency alert systems (K4); evaluate how compiler optimization improves reliability in healthcare applications (K5).*

Task (i): Intermediate Code for Patient Condition Monitoring Logic (K3)

The monitoring loop evaluates a critical vital-sign condition (abnormal heart rate) using an OR expression and triggers an alarm procedure when out of range:

```
if (heartRate < 60 || heartRate > 100) then
    alertFlag = true;
    triggerAlarm(patientId);
else
    alertFlag = false;
```

For the OR operator (B → B1 || M B2), the SDT scheme backpatches B1.falselist (not truelist, as in AND) to the marker before B2 — i.e., B2 is only evaluated when B1 is false, implementing correct short-circuit OR semantics. This ordering is what Task (ii) below explains is safety-critical. The TAC is:

Op	Arg1	Arg2	Result
<	heartRate	60	t1
if t1	goto		Lalarm (106)
			(fall through)
>	heartRate	100	L2: t2
if t2	goto		Lalarm (106)
goto			Lnormal (109)
=	true		Lalarm: alertFlag
param	patientId		
call	triggerAlarm	1	
goto			Lnext
=	false		Lnormal: alertFlag

Both t1 and t2 backpatch their true-branch to the same Lalarm target (merge(B1.truelist, B2.truelist) → Lalarm) so only one alarm-triggering block is generated regardless of which vital-sign threshold was breached.

Architecture / Flow Diagram

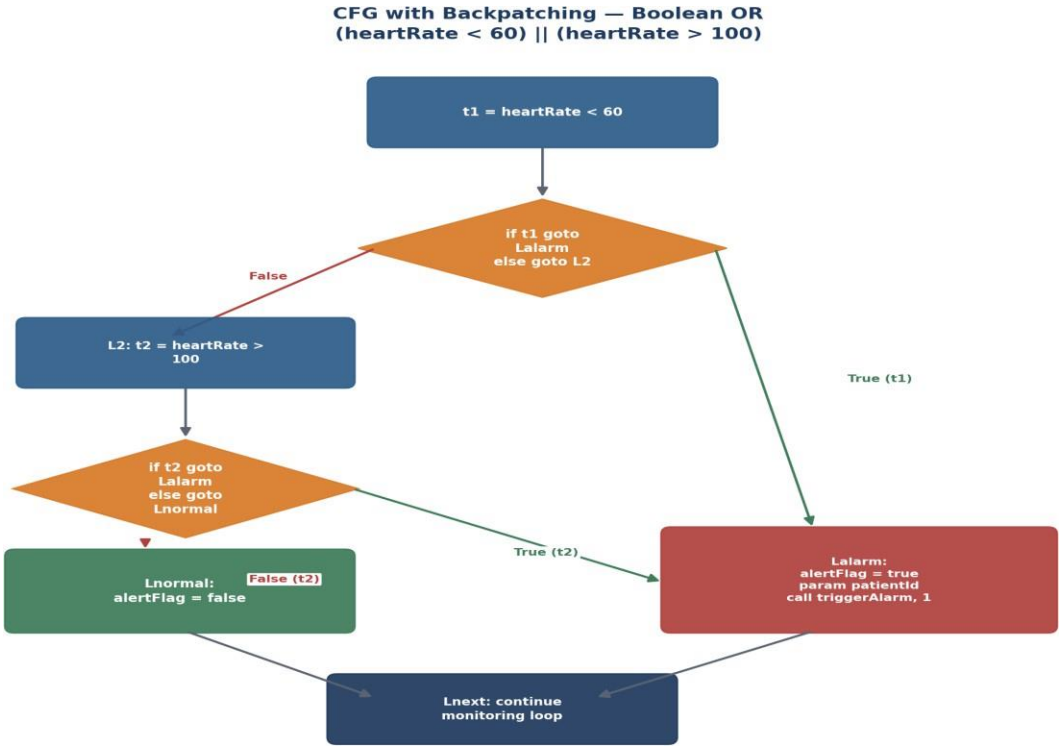

Figure 4. Backpatched control-flow graph for the OR condition that triggers the emergency alarm procedure.

Task (ii): Significance of Backpatching in Emergency Alert Systems (K4)

The scenario states that improper backpatching in conditional branching caused delayed alarm generation — this maps directly onto specific backpatching failure modes and why correct backpatching matters here more than almost anywhere else:

- **Correct short-circuit semantics for OR:** Backpatching B1.falselist (not truelist) to B2's start is what makes the alarm trigger the instant the first condition (heartRate < 60) is true, without needlessly evaluating the second sensor read — saving critical microseconds and avoiding a possible additional sensor-read stall in the alarm path.
- **Single merged alarm target:** Using merge() to combine both sub-conditions' true-lists into one backpatch target ensures the alarm procedure is generated and patched exactly once, avoiding the risk of two divergent (and potentially inconsistent) alarm code paths that a naive, non-backpatched translation might produce.

- **One-pass reliability under time pressure:** Backpatching lets the compiler generate correct jump targets in a single forward scan, without needing a second pass to resolve labels — important because monitoring-software updates (e.g., adjusting alert thresholds) must be recompiled and redeployed quickly and correctly, and a slow or bug-prone two-pass scheme increases the window during which faulty code could run.
- **Consequence of failure:** If a backpatch target is left unpatched or patched to the wrong quad (exactly the defect described in the scenario), the alarm branch may never execute, or may execute late after an unrelated jump — directly causing the delayed alarm generation described, with potentially life-threatening consequences.

Task (iii): Compiler Optimization Improving Reliability in Healthcare Applications (K5)

Beyond correctness, compiler optimizations materially improve the reliability and timeliness of safety-critical monitoring software:

- **Reduced execution time per monitoring cycle:** Dead-code elimination and constant folding shrink the vital-sign-check loop so it comfortably fits within the sensor's required polling interval, reducing the risk of a missed reading during a critical episode.
- **Common sub-expression elimination:** Avoids redundant re-reads or re-computation of the same sensor value across multiple threshold checks (heart rate, oxygen saturation, blood pressure), reducing both latency and the chance of using a stale value in one check versus another.
- **Register allocation & reduced interrupt latency:** Keeping critical monitoring variables in registers minimizes the time the interrupt handler takes to evaluate alarm conditions, directly reducing alarm-response jitter.
- **Verification-friendly optimized IR:** A clean, optimized intermediate representation is easier to formally analyze and test against healthcare software standards (e.g., IEC 62304), supporting the certification process that safety-critical medical software must pass.
- **Trade-off discipline:** As with the traffic-control case, optimization must be tuned for predictable worst-case latency rather than best-case throughput — an optimization that speeds up the average case but occasionally causes a longer worst-case delay would be unacceptable in an emergency-alert path.

Question 5 — Online Airline Reservation System (10 Marks)

Tasks: *Develop intermediate code for ticket booking and seat allocation operations (K3); explain how procedure calls support modular reservation management (K4); analyze the role of intermediate languages in distributed airline reservation systems (K5).*

Task (i): Intermediate Code for Ticket Booking & Seat Allocation (K3)

The booking workflow checks seat availability and, if free, assigns the seat and confirms the booking; otherwise the passenger is waitlisted:

```
procedure bookSeat(flightId, seatNo, passengerName)
    if (isSeatAvailable(flightId, seatNo)) then
        assignSeat(flightId, seatNo, passengerName);
        status = "CONFIRMED";
    else
        status = "WAITLIST";
    end
```

Procedure calls generate a standard param / call TAC sequence, pushing arguments onto (or into) the callee's activation record before transferring control. The generated TAC is:

Op	Arg1	Arg2	Result
param	flightId		
param	seatNo		
call	isSeatAvailable	2	t1
if t1	goto		Lbook (108)
=	"WAITLIST"		status
goto			Lend
param	flightId		Lbook:
param	seatNo		
param	passengerName		
call	assignSeat	3	
=	"CONFIRMED"		status

The integer after each call quad (“2”, “3”) records the argument count, which the code generator uses to correctly size and tear down the activation record after the call returns

— essential when bookSeat may be invoked concurrently by thousands of simultaneous booking requests.

Architecture / Flow Diagram

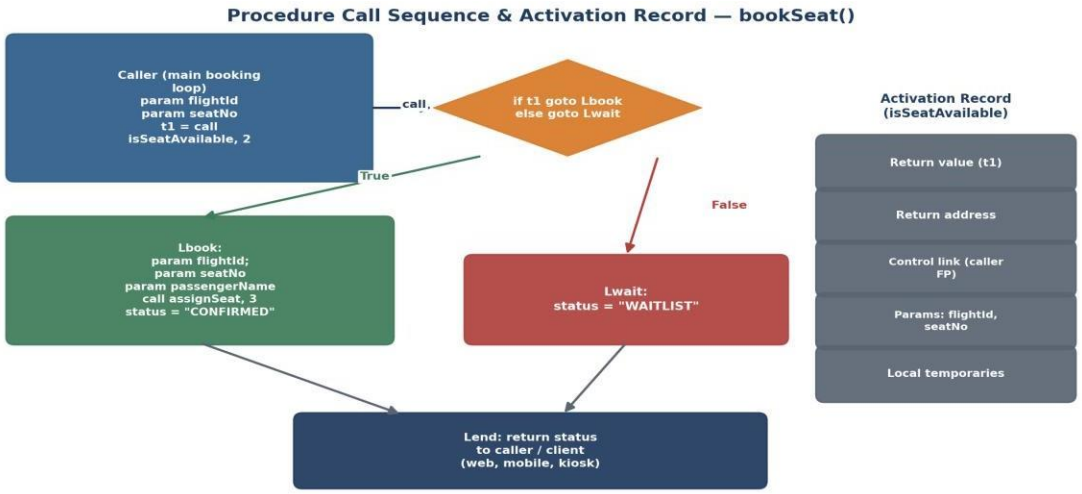

Figure 5. Procedure-call sequence and activation-record layout for the bookSeat() operation.

Task (ii): Procedure Calls Support Modular Reservation Management (K4)

Structuring booking logic as discrete procedures (isSeatAvailable, assignSeat, processPayment, cancelBooking, …) rather than one monolithic script is what makes the reservation platform maintainable at airline scale:

- **Separation of concerns:** Seat-availability checking, seat assignment, and payment confirmation are independently testable and independently optimizable units, matching how the scenario describes errors specifically in intermediate code generation affecting seat allocation and payment confirmation — modular procedures isolate and simplify fixing such defects.
- **Well-defined activation records:** Each call passes exactly the parameters it needs (flightId, seatNo, passengerName) and returns a clear result (t1, status), via a compiler-managed activation record containing the return value, return address, control link, parameters, and locals — preventing state from one booking request leaking into another.

- **Reusability across channels:** The same compiled bookSeat() procedure is invoked identically whether the request originates from the airline's website, mobile app, or airport kiosk, ensuring consistent business rules across every distribution channel.
- **Safe concurrency:** Because each call creates its own activation record (typically on a per-request stack or coroutine frame), thousands of simultaneous booking requests can invoke bookSeat() concurrently without interfering with each other's temporaries or parameters.
- **Incremental optimization:** The compiler can apply inlining for small, frequently-called helper procedures (e.g., a simple seat-status lookup) while keeping larger procedures (payment processing) as true calls — a modular design gives the optimizer this flexibility.

Task (iii): Intermediate Languages in Distributed Airline Reservation Systems (K5)

Airline reservation platforms typically run as distributed services across multiple data centres (regional booking servers, global inventory/seat-map services, payment gateways). An intermediate representation is central to making this distributed architecture reliable and consistent:

- **Consistent business logic across nodes:** Compiling the booking script once to a machine-independent IR, then generating target code for each server type (booking-engine servers, mobile backends, airport kiosks) guarantees identical seat-allocation and payment logic everywhere, preventing the scenario's described errors where inconsistent code generation affected seat allocation differently across parts of the system.
- **Retargetable back ends:** As the airline's distributed infrastructure evolves (e.g., migrating booking servers to new cloud hardware or container platforms), only the IR-to-target back end needs to change; the front end and all IR-level optimizations remain valid investment.
- **Enables safe, machine-independent optimization before distribution:** Optimizations (redundant seat-check elimination, inlining of small validation procedures) are applied once on the IR before it is distributed/deployed to any node, so every regional server benefits equally and consistently, rather than optimizing separately (and potentially inconsistently) per platform.
- **Facilitates large-scale, high-concurrency processing:** A clean IR with explicit procedure-call and activation-record semantics (as generated above) gives the runtime

a precise, analyzable structure for managing thousands of simultaneous distributed booking transactions reliably — directly addressing the scenario's goal of handling thousands of simultaneous booking requests.

- **Supports formal/testable correctness:** A simpler, regular IR is easier to test and simulate under distributed-systems conditions (e.g., network partitions, retries) before generating final deployable code, reducing the risk of the payment-confirmation and seat-allocation errors described in the scenario.